

Foundations of Object-Oriented Programming and Data Structures: A Formal Mathematical Analysis

Abstract—This paper presents a formal, mathematical abstraction of Object-Oriented Programming (OOP) paradigms and essential data structures. By transitioning from descriptive heuristics to rigorous mathematical formulations, we explore the core mechanics of classes, objects, and memory management through set theory and mapping functions. Furthermore, the paper provides a formal definition of object duplication and a structural, graph-theoretic analysis of Linked Lists.



1 Introduction

The adoption of Object-Oriented Programming (OOP) requires a paradigm shift towards encapsulated states and behaviors. Formally, OOP models a software system as a set of interacting discrete entities. Unlike procedural paradigms modeled as a sequence of functions $\mathcal{F}$ applied to a global state space S , OOP defines isolated, partitioned state spaces S_i mapped to specific behavioral operations M_i , maximizing modularity and cohesion.

2 Core Concepts: Classes and Objects

We formally define the pillars of OOP, Classes and Objects, utilizing set theory.

- **Class ($\mathcal{C}$):** A class is defined as an ordered pair $\mathcal{C} = (A, M)$, where $A = \{a_1, a_2, \dots, a_n\}$ represents a finite set of attributes (state variables) and $M = \{m_1, m_2, \dots, m_k\}$ represents a finite set of methods (functions).
- **Object (o):** An object is a specific instance of a class. The instantiation is a surjective mapping function $I : \mathcal{C} \rightarrow O$, where O is the set of all objects. Thus, $o \in O$, and o encapsulates a concrete state vector $\vec{v} \in \text{dom}(a_1) \times \text{dom}(a_2) \times \dots \times \text{dom}(a_n)$.

To reference its own internal state space, an object utilizes an identity mapping operator, denoted conventionally as this or self.

3 Memory Management: Stack vs. Heap Spaces

Memory architecture can be modeled as two disjoint sets of addressable spaces: the Stack (S) and the Heap (H).

- **Heap Memory (H):** A dynamically allocated memory space where the state data of an object o resides. Instantiation reserves a contiguous memory block $h \in H$.
- **Stack Memory (S):** A structured memory space that stores local reference variables. A reference variable $r \in S$ does not store the object state directly. Instead, it acts as a pointer mapping, defined by a function $f : S \rightarrow H \cup \{\emptyset\}$, where $f(r) = h$, thereby locating the object o in the Heap.

4 Object Instantiation and Constructors

Constructors are initialization functions, $C : \text{dom}(A) \rightarrow O$, invoked during the instantiation I .

- **Default Constructor (C_{def}):** Maps an object's state vector to a defined zero-element or null-element $\vec{0}$.
- **Parameterized Constructor (C_{param}):** An injective function that maps a provided parameter vector $\vec{p}$ directly to the object's initial state vector $\vec{v}$.

5 Object Duplication: Formalizing Shallow vs. Deep Copy

Let o_1 be an object residing at heap address $h_1 \in H$, referenced by $r_1 \in S$ such that $f(r_1) = h_1$.

- **Shallow Copy:** An assignment operator that maps a new reference $r_2 \in S$ to the exact same heap address. Formally, $f(r_2) = f(r_1) = h_1$. Thus, any state transformation T applied via r_1 or r_2 mutates the same underlying object o_1 .
- **Deep Copy:** An operation that allocates a new heap address $h_2 \in H$ (where $h_1 \neq h_2$) for a new object o_2 . An element-wise state cloning is performed: $\forall a_i \in A, o_2.a_i = o_1.a_i$. Finally, the new reference maps to the new address: $f(r_2) = h_2$. The subsets representing o_1 and o_2 become completely disjoint.

6 Memory Optimization: Static Data Members

Let the cardinality of instantiated objects of class $\mathcal{C}$ be $|O| = N$. For instance variables, the total memory allocated is $\mathcal{O}(N \cdot |A|)$. To optimize space complexity, we define a subset of attributes $A_{static} \subseteq A$. A static member $a_s \in A_{static}$ is mapped identically for all $o \in \mathcal{C}$. Thus, a_s is bound to the class definition $\mathcal{C}$ itself rather than to individual objects, reducing the space complexity of a_s from $\mathcal{O}(N)$ to $\mathcal{O}(1)$.

7 Linear Data Structures: Linked Lists

We analyze a Linked List L as a directed acyclic graph (DAG) composed of a set of vertices V (nodes). A node $N_i \in V$ is formalized as a tuple (d_i, p_i) , where:

- 1) $d_i \in \mathcal{D}$ is the data element from domain $\mathcal{D}$.
- 2) $p_i : N_i \rightarrow N_{i+1}$ is a directed edge mapping the current node to the subsequent node.

7.1 Traversal and Operations

The sequence is bounded by a head node N_0 and a tail node N_k , where $p_k \rightarrow \emptyset$ (null). Traversal requires applying a composition of mappings. To reach the K -th node N_k from N_0 , we apply the mapping function K times:

$$N_k = p^{(K)}(N_0) = \underbrace{p \circ p \circ \dots \circ p}_{K \text{ times}}(N_0)$$

In algorithmic implementation, this sequential mapping is executed via iterative pointer reassignment ($\text{temp} = \text{temp.next}$).

8 Conclusion

Object-Oriented Programming and its underlying data structures can be robustly defined through mathematical formalisms. Modeling classes as sets of attributes and methods, references as mappings between Stack and Heap domains, and linear structures as directed graphs provides a rigorous theoretical foundation. This mathematical approach is essential for analyzing the computational complexity, memory efficiency, and structural integrity of modern software architectures.